PKCERT AI & Software Development Internship

Task 22 – Feedforward Neural Networks on MNIST, Regularization & GPU-Accelerated Training

---

Objective:

Consolidate every neural-network concept covered so far — perceptrons, activation functions, forward propagation, and backpropagation — into a complete, working feedforward network trained on MNIST (or a comparable image dataset). Extend that network with three core regularization techniques (dropout, batch normalization, early stopping), then move training onto a GPU-accelerated notebook (Google Colab or Kaggle) and rigorously benchmark the difference it makes.

Total Marks: 100

This task assumes full command of Task 16's material (perceptron mechanics, activation function derivatives, and the manual forward/backward derivation). Several sections below will not re-explain those fundamentals — you are expected to apply them correctly and to catch your own mistakes if a gradient or shape doesn't check out.

Part A – Conceptual Refresher & Integration (10 Marks)

1. Without referring back to Task 16, re-derive from memory the backpropagation gradients for a 2-hidden-layer network using ReLU activations in the hidden layers and softmax with cross-entropy loss at the output. Show your work; do not just state the final formulas.
2. Explain precisely why softmax is paired with cross-entropy loss rather than mean squared error, referencing the simplified gradient (predicted − actual) that results at the output layer.
3. In 200–300 words, explain how a perceptron, an activation function, and the backpropagation algorithm compose into a trainable feedforward network — written as if explaining it to a junior intern who only knows the perceptron.

Part B – Feedforward Neural Network on MNIST (20 Marks)

4. Load the MNIST dataset (or, if you want an extra challenge, Fashion-MNIST) and build a feedforward network with at least two hidden layers.
5. Implement the network twice: once from scratch using only NumPy (forward pass, backward pass, and mini-batch gradient descent — no autograd), and once using a framework of your choice (PyTorch, TensorFlow, or Keras).
6. Confirm both implementations reach comparable test accuracy (report the gap and explain any discrepancy — do not paper over it).
7. Plot training and validation loss/accuracy curves for both implementations on the same figure for direct comparison.
8. Report a full confusion matrix and identify which digit/class pairs are most frequently confused, with a brief hypothesis as to why.

Part C – Regularization Techniques: Dropout, Batch Normalization & Early Stopping (25 Marks)

9. Implement inverted dropout from scratch in your NumPy network: correct forward-pass masking and scaling, and a backward pass that correctly propagates gradients only through active units.

10. Implement batch normalization from scratch in your NumPy network: forward pass (batch mean/variance, normalization, learnable scale/shift), backward pass (correct gradients w.r.t. inputs, scale, and shift), and running statistics for inference-time use.
11. Implement early stopping based on validation loss with a configurable patience parameter, and restore the best-performing weights rather than the final-epoch weights.
12. Run a controlled ablation study with at least four configurations — baseline (no regularization), dropout only, batch norm only, and dropout + batch norm + early stopping combined — using identical architecture, data split, and initialization seed for each. Tabulate train/validation/test accuracy and the train–test accuracy gap for each configuration.
13. Mathematically justify why dropout approximates training an ensemble of sub-networks, and explain — grounded in the actual mechanism (reducing internal covariate shift and smoothing the loss landscape), not marketing language — why batch normalization tends to stabilize and accelerate training.
14. Identify a hyperparameter mistake that would make dropout or batch normalization actively harmful (e.g., wrong placement relative to the activation, dropout at inference time, mismatched train/eval mode), and demonstrate its effect empirically.

Part D – GPU Basics: Colab / Kaggle Accelerated Training (20 Marks)

15. Move your framework-based (Part B) implementation to a GPU-enabled runtime using Google Colab or Kaggle Notebooks, and confirm the GPU is actually being used (device check, not just an assumption).
16. Benchmark identical training runs (same architecture, batch size, epochs, seed) on CPU versus GPU. Report wall-clock time for both and compute the speedup factor.
17. Explain, mechanistically, why GPUs accelerate neural network training — parallel matrix multiplication across many cores and high memory bandwidth — rather than repeating a generic 'GPUs are faster' claim.
18. Enable mixed-precision training (fp16/bfloat16) if your framework and the notebook's GPU support it, and report the effect on training time and memory usage compared to full precision.
19. Discuss at least two practical constraints of free-tier GPU notebooks (e.g., session timeouts, usage quotas, disconnections) and the specific mitigation you used (e.g., periodic checkpointing to Drive/Kaggle datasets, resumable training loop).

Part E – Hardcore Integration Challenge (15 Marks)

20. Combine everything above into one final pipeline: the from-scratch NumPy network with dropout, batch normalization, and early stopping, trained via mini-batch gradient descent — and a matching framework implementation with the same regularization stack, trained on GPU.
21. Statistically compare the two final models' predictions (e.g., agreement rate on the test set, or a paired comparison of per-class accuracy) and explain any meaningful divergence rather than dismissing it.
22. Stress-test robustness: evaluate both final models on a corrupted test set (e.g., MNIST with added Gaussian noise, or a small random-rotation augmentation) and report the accuracy drop. Discuss whether your regularization choices helped or hurt robustness to this shift.
23. Given a fixed compute budget (state a self-imposed limit, e.g., 10 minutes of GPU time), decide and justify how you would allocate it across architecture size, regularization strength, and training epochs to maximize test accuracy.

Part F – Analysis & Documentation (10 Marks)

24. Save the final trained model(s) (weights/parameters) using pickle, joblib, or the framework's native format.
25. Summarize the complete pipeline: architecture, regularization stack, and training environment (CPU vs GPU), with justification for each choice.

26. Summarize the ablation study and benchmarking results in a single consolidated table.
27. Discuss at least two challenges faced during this task and how you diagnosed and resolved them (a vague 'it was hard' does not qualify — describe the actual debugging process).

Assessment Breakdown

Assessment Area	Marks
Conceptual Refresher & Integration	10
Feedforward Neural Network on MNIST	20
Regularization: Dropout, Batch Norm & Early Stopping	25
GPU Basics: Colab / Kaggle Accelerated Training	20
Hardcore Integration Challenge	15
Analysis & Documentation	10
Total	100

Note: For the from-scratch components in Parts A, B, and C, use of high-level automatic-differentiation frameworks is not permitted — the intent is to prove you understand the underlying mathematics well enough to implement and debug it yourself. Frameworks are required (not optional) for the GPU-accelerated components in Parts D and E.